

Task 1 : Create a client-server application using Named Pipes (FIFOs). The client sends messages and the server processes and responds to them. Analyze FIFO behavior when multiple clients communicate simultaneously.

Code:

```
GNU nano 7.2
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <fcntl.h>
#include <unistd.h>
#include <sys/stat.h>

#define FIFO1 "client_to_server"
#define FIFO2 "server_to_client"

int main() {
    char message[100];
    int fd_read, fd_write;

    mkfifo(FIFO1, 0666);
    mkfifo(FIFO2, 0666);

    printf("Server started. Waiting for client...\n");

    fd_read = open(FIFO1, O_RDONLY);

    read(fd_read, message, sizeof(message));
    close(fd_read);

    printf("Server received: %s\n", message);

    fd_write = open(FIFO2, O_WRONLY);

    char response[] = "Message received by server.";
    write(fd_write, response, strlen(response) + 1);

    close(fd_write);

    printf("Server response sent.\n");

    unlink(FIFO1);
    unlink(FIFO2);

    return 0;
}
```

Output:

```
nanopb-0.1.0/Makefile.in:24:
gopinath@GOPINATH:~$ nano p-6.c
gopinath@GOPINATH:~$ gcc p-6.c
gopinath@GOPINATH:~$ ./a.out
Server started. Waiting for client...
```

code for client :

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <fcntl.h>
#include <unistd.h>

#define FIF01 "client_to_server"
#define FIF02 "server_to_client"

int main() {
    char message[100];
    char response[100];
    int fd_write, fd_read;

    printf("Enter message: ");
    fgets(message, sizeof(message), stdin);

    message[strcspn(message, "\n")] = '\0';

    fd_write = open(FIF01, O_WRONLY);

    write(fd_write, message, strlen(message) + 1);
    close(fd_write);

    fd_read = open(FIF02, O_RDONLY);

    read(fd_read, response, sizeof(response));
    close(fd_read);

    printf("Server response: %s\n", response);

    return 0;
}
```

Output:

```
Server started. Waiting for client...
Server received: hello server
Server response sent.
```

Observation: To implement client-server communication using Named Pipes (FIFOs). □ how the client sends messages through the FIFO and how the server reads, processes, and sends a response back.

- mkfifo(), open(), read(), and write() are used for inter-process communication and FIFO behavior changes when multiple clients communicate with the server.

Task2: : Create a POSIX signal handling program that captures SIGINT, SIGTERM, and SIGUSR1. Demonstrate asynchronous event handling and explain the role of signal handlers.

Code:

```
GNU nano 8.7.1
#include <stdio.h>
#include <stdlib.h>
#include <signal.h>
#include <unistd.h>

void signal_handler(int signal) {
    if (signal == SIGINT) {
        printf("\nSIGINT received (Ctrl+C).\n");
    }
    else if (signal == SIGTERM) {
        printf("\nSIGTERM received.\n");
    }
    else if (signal == SIGUSR1) {
        printf("\nSIGUSR1 received.\n");
    }

    fflush(stdout);
}

int main() {
    signal(SIGINT, signal_handler);
    signal(SIGTERM, signal_handler);
    signal(SIGUSR1, signal_handler);

    printf("Signal handling program started.\n");
    printf("PID: %d\n", getpid());
    printf("Waiting for signals...\n");

    while (1) {
        pause();
    }

    return 0;
}
```

Output:

```
gopinath@GOPINATH:~$ gcc p62.c
gopinath@GOPINATH:~$ ./a.out
Signal handling program started.
PID: 648
Waiting for signals...          ^C
                                SIGINT received (Ctrl+C).
```

```
SIGUSR1 received.
```

```
SIGTERM received.
```

Observation: To handle POSIX signals such as SIGINT, SIGTERM, and SIGUSR1 in C.

How a signal handler responds to signals sent to a running process. To use the kill command to send signals using a process ID (PID) and how pause() allows a process to wait for signals.